Local Search Methods

Hill Climbing and Gradient Descent

Contents

1 Introduction to Local Search **2**
- 1.1 Key Characteristics of Local Search . . . 2
- 1.2 When to Use Local Search . . . 2
- 1.3 The Optimization Landscape . . . 2

2 Hill Climbing Algorithm **4**
- 2.1 The Basic Algorithm . . . 4
- 2.2 Line-by-Line Explanation . . . 4
  - 2.2.1 Line 1: Function Declaration . . . 4
  - 2.2.2 Line 2: Initialize Starting Node . . . 4
  - 2.2.3 Line 3: Main Loop . . . 4
  - 2.2.4 Line 4: Select Best Successor . . . 5
  - 2.2.5 Lines 5-6: Termination Check . . . 5
  - 2.2.6 Line 7: Move to Best Successor . . . 5
- 2.3 Alternative Name: Gradient Descent . . . 5
- 2.4 Use in Optimization Problems . . . 6

3 Properties of Hill Climbing **7**
- 3.1 Completeness . . . 7
- 3.2 Optimality . . . 7
- 3.3 Time Complexity . . . 8
- 3.4 Space Complexity . . . 8
- 3.5 Summary of Properties . . . 8

4 Example: 8-Queens Problem **9**
- 4.1 Problem Formulation . . . 9
- 4.2 Example State and Heuristic Calculation . . . 9
- 4.3 Heuristic Values for Neighbors . . . 9
- 4.4 Performance of Basic Hill Climbing . . . 10

5 Local Minima: The Main Challenge **11**
- 5.1 What is a Local Minimum? . . . 11
- 5.2 Example: Local Minimum in 8-Queens . . . 11

6 Difficulties in Local Search **12**
- 6.1 Local Minima . . . 12
- 6.2 Plateaus . . . 12
- 6.3 Ridges and Valleys . . . 13

7 Strategies for Improving Local Search **14**
7.1 Random Restart . . . 14
7.2 Random Walk . . . 15
7.3 Sideways Moves . . . 16
7.4 Parameter Configuration . . . 17

8 Hill Climbing with Sideways Moves: 8-Queens Example **18**
8.1 Configuration . . . 18
8.2 Performance Comparison . . . 18
8.3 Exploration vs. Exploitation Trade-off . . . 19

9 Can Local Minima Occur in Route Planning? **20**
9.1 Similar Issues in Grid-Based Path Planning . . . 21

10 Comparison: Local Search vs. Systematic Search **22**
10.1 Key Differences . . . 22
10.2 When to Use Each Approach . . . 22
10.3 Hybrid Approaches . . . 23

11 Advanced Local Search Methods **24**
11.1 Simulated Annealing . . . 24
11.2 Tabu Search . . . 24
11.3 Genetic Algorithms and Population-Based Methods . . . 25

12 Summary and Key Takeaways **26**
12.1 Main Concepts . . . 26
12.2 Key Difficulties . . . 26
12.3 Improvement Strategies . . . 26
12.4 Practical Success . . . 27
12.5 Comparison with Systematic Search . . . 27
12.6 Final Thoughts . . . 27

1 Introduction to Local Search

Local search represents a fundamentally different approach to search compared to systematic methods like A* or breadth-first search. Instead of maintaining a complete frontier of unexplored states and systematically exploring the state space, local search makes decisions based solely on the immediate neighborhood of the current state.

The Computer's "Nose"

Sometimes, "going where your nose leads you" works quite well. But what is the computer's "nose"?
Answer: The heuristic function h evaluated on immediate neighbor states.
Local search takes decisions based on the h values of immediate neighbor states, moving to whichever neighbor appears most promising according to the heuristic.

1.1 Key Characteristics of Local Search

- **Memory Efficiency**: Keeps only one (or a few) search nodes in memory at any time
- **Greedy Decision Making**: Makes decisions based only on immediate neighbors
- **No Backtracking**: Once a decision is made, there is typically no going back
- **Incomplete**: Not guaranteed to find a solution even if one exists
- **Not Optimal**: Even if a solution is found, no guarantee it is optimal
- **Fast and Practical**: Despite lack of guarantees, often works well in practice for large-scale optimization problems

1.2 When to Use Local Search

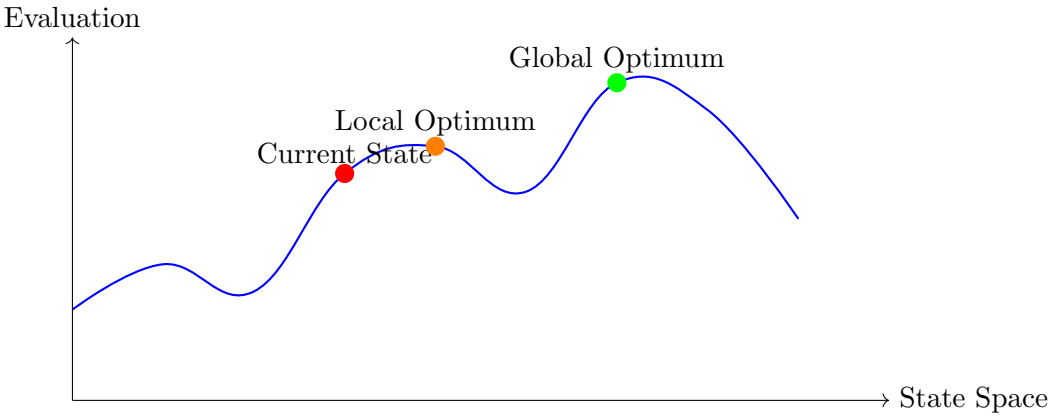
Local search is particularly appropriate when:

1. **Optimization Problems**: All states are valid solutions, and we want to find a good (not necessarily optimal) one
2. **Large State Spaces**: Memory constraints make systematic search infeasible
3. **Good-Enough Solutions**: We are satisfied with a reasonably good solution rather than the optimal one
4. **Time Constraints**: We need a solution quickly
5. **Flexible Neighborhoods**: We can define the search neighborhood freely

1.3 The Optimization Landscape

We can visualize the search space as a landscape where:

- The horizontal dimensions represent different states
- The vertical dimension represents the evaluation function (heuristic value)
- Peaks represent local maxima (or minima if we're minimizing)
- The global peak represents the optimal solution
- Local search navigates this landscape by following the gradient


Evaluation
Global Optimum
Local Optimum
Current State
State Space

2 Hill Climbing Algorithm

Hill climbing is the most basic local search algorithm. The name comes from the metaphor of climbing a hill in foggy conditions: you can only see your immediate surroundings, so you keep stepping uphill until you reach a point where all directions go down.

2.1 The Basic Algorithm

Hill Climbing Algorithm (Following Lecture 3

```
1: function HillClimbing(heuristic h) returns a path
2:     n ← InitNode()
3:     while true do
4:         n′ ← among successors n′ of n with minimal h(n′), select one randomly
5:         if h(n′) ≥ h(n) then
6:             return ExtractSolution(n)
7:         end if
8:         n ← n′
9:     end while
10: end function
```

2.2 Line-by-Line Explanation

2.2.1 Line 1: Function Declaration

`function HillClimbing(heuristic h) returns a path`

Purpose: Declares the hill climbing function that takes a heuristic as parameter.
Input:

- A heuristic function h that estimates the cost/quality of any state

Output:

- A path (sequence of states/actions) leading to a local minimum

Note: Unlike systematic search algorithms, there is no "unsolvable" return value. The algorithm always returns when it reaches a local minimum.

2.2.2 Line 2: Initialize Starting Node

`n ← InitNode()`

Purpose: Creates the initial search node corresponding to the start state.
Details:

- $n.State$ is set to the initial state I
- $n.Pred =$ `null` (no predecessor)
- $n.Action =$ `null` (no action led here)

Important Difference from Systematic Search: We do not maintain an open list or closed set. We only keep track of the current node n.

2.2.3 Line 3: Main Loop

`while true do`

Purpose: Continues the search indefinitely until a stopping condition is met (inside the loop).
Termination: The loop terminates when we reach a local minimum (line 5).

2.2.4 Line 4: Select Best Successor

```
n' ← among successors n' of n with minimal h(n'), select one randomly
```

Purpose: Finds the successor state with the best (lowest) heuristic value.
Key Points:

- We evaluate h for all successors of the current state
- We select the successor(s) with minimal $h(n')$
- If there are multiple successors with the same minimal value (ties), we select one randomly
- This is the "greedy" step: we always move in the direction that looks best right now

Notation: "Successors n' of n" means all nodes reachable by applying one action from state $n.State$.

2.2.5 Lines 5-6: Termination Check

```
if h(n')  h(n) then
    return ExtractSolution(n)
```

Purpose: Checks if we have reached a local minimum.
Condition: If the best successor is no better than (or worse than) the current state, we have reached a local minimum.
Why This Matters:

- If $h(n') > h(n)$: All neighbors are strictly worse, so n is a strict local minimum
- If $h(n') = h(n)$: All neighbors are the same or worse, so n is on a plateau or a local minimum
- In either case, we cannot improve by moving to any neighbor, so we stop

Return Value: $\textsc{ExtractSolution}(n)$ reconstructs the path from the initial node to n by following predecessor pointers.

2.2.6 Line 7: Move to Best Successor

```
n ← n'
```

Purpose: Updates the current node to the best successor.
Effect: We "move" to the best neighbor state and repeat the process.
Memory Note: We discard the old node n (except via the predecessor pointer in n'). We never keep multiple nodes in memory simultaneously.

2.3 Alternative Name: Gradient Descent

Gradient Descent

Hill climbing is also called **gradient descent** when we think of minimizing the heuristic value (descending into a valley) rather than maximizing it (climbing a hill).
Terminology:

- **Hill Climbing**: Maximizing an objective function
- **Gradient Descent**: Minimizing a cost function

In our context, since h represents estimated cost (lower is better), we are technically doing gradient descent. However, the term "hill climbing" is standard in AI search literature.

2.4 Use in Optimization Problems

Optimization Problem Context

Hill climbing is often used in optimization problems where:

- All "states" are feasible solutions
- We can choose the search neighborhood ("child nodes") freely
- We want to return just $n.State$ (the solution itself) rather than the path to n

Example: In the 8-Queens problem:

- **States**: Configurations with one queen per column
- **Feasible**: All states have 8 queens on the board
- **Heuristic**: Number of pairs of queens attacking each other
- **Goal**: Find a state with $h = 0$ (no attacks)
- **Neighborhood**: Move any queen within its column

3 Properties of Hill Climbing

3.1 Completeness

Completeness

Question: Is hill climbing complete? That is, if a solution exists, will hill climbing find it?
Answer: **No**.
Hill climbing is **not complete**. The search ends when no more immediate improvements can be made, which occurs at a local minimum. A local minimum is not guaranteed to be a solution (global minimum).

Why Not Complete?
Consider a state s where:

- $h(s) > 0$ (not a goal state)
- For all successors s' of s: $h(s') \geq h(s)$

In this case:

- Hill climbing will stop at s
- s is a local minimum but not a solution
- Even if a goal state is reachable from s, hill climbing will not find it

3.2 Optimality

Optimality

Question: Is hill climbing optimal? That is, does it find the best possible solution?
Answer: **No**.
Hill climbing is **not optimal**. It stops at a local minimum, which may be far from the global optimum.

Why Not Optimal?

- Hill climbing makes locally optimal decisions (always move to the best neighbor)
- But locally optimal decisions do not guarantee globally optimal results
- The algorithm may get stuck in a local minimum that is much worse than the global minimum
- There is no backtracking mechanism to escape local minima

3.3 Time Complexity

Time Complexity

Worst-Case Bound: The size of the state space $|S|$ is an upper bound on the number of steps.
Why?: Since $h(n)$ strictly decreases with each move (except at plateaus), we cannot visit the same state twice. Therefore, we can visit at most $|S|$ states.
Important Notes:

1. This bound is **huge**: The state space is typically exponentially large
2. This bound applies to a **single run** of hill climbing
3. A single run **typically does not find** a solution (due to local minima)
4. In practice, we run hill climbing many times with different random starting points

3.4 Space Complexity

Space Complexity

Space Complexity: $O(b)$ where b is the branching factor.
Why?: At any moment, we only keep in memory:

- The current node n
- The successors of n (temporarily, while evaluating them)

This is a **major advantage** of hill climbing over systematic search methods like A*, which must keep all generated nodes in memory.

3.5 Summary of Properties

Property	**Hill Climbing**
Complete	No
Optimal	No
Time Complexity	$O(\|S\|)$ per run (huge, impractical bound)
Space Complexity	$O(b)$ (excellent!)

Trade-off

Hill climbing trades **completeness and optimality** for **memory efficiency** and **speed**.
This trade-off is often worthwhile in practice for:

- Very large state spaces where systematic search is infeasible
- Problems where a good solution is acceptable (don't need optimal)
- Time-critical applications

4 Example: 8-Queens Problem

The 8-Queens problem is a classic example used to illustrate local search methods.

4.1 Problem Formulation

8-Queens Problem

Goal: Place 8 queens on a chess board such that no two queens attack each other.
Rules: A queen attacks any piece on the same row, column, or diagonal.
State Representation: One queen per column, each queen identified by its row position.

- A state is a tuple $(r_1, r_2, \ldots, r_8)$ where $r_i \in \{1, 2, \ldots, 8\}$
- r_i is the row position of the queen in column i

Heuristic: Number of pairs of queens attacking each other.

- A solution has $h = 0$ (no attacks)
- We want to minimize h

Neighborhood: From any state, we can move any one queen to any position within its column.

- Each queen can move to 7 other positions in its column
- Total of $8 \times 7 = 56$ neighbors per state

Initial State: Random placement of queens (one per column).

4.2 Example State and Heuristic Calculation

Consider a state with queens at positions: (1,2,3,4,5,6,7,8) (diagonal line).

Calculating Attacks:

- Row attacks: 0 (all queens in different rows)
- Column attacks: 0 (by construction, one queen per column)
- Diagonal attacks: Many! Each queen attacks multiple others on the diagonal

For the diagonal configuration, every pair of queens separated by distance d along the diagonal attacks each other. The total number of attacking pairs is $\binom{8}{2} = 28$.

4.3 Heuristic Values for Neighbors

At each state, we evaluate the heuristic for all 56 neighbors (7 moves for each of 8 queens). The heuristic value tells us how many pairs of queens would attack each other if we made that move.

Heuristic Grid Example

From the lecture slides, we see an example state showing heuristic values for all possible moves:

18	**12**	14	13	13	**12**	14	14
14	16	13	15	**12**	14	**12**	16
14	**12**	18	13	15	**12**	14	14
15	14	14	Q	13	16	13	16
Q	14	17	15	Q	14	16	16
17	Q	16	18	15	Q	15	Q
18	14	Q	15	15	14	Q	16
14	14	13	17	**12**	14	**12**	18

The Q symbols show current queen positions. The numbers show the heuristic value (number of attacks) if we moved a queen to that position. The bold values (**12**) are the minimum, indicating the best moves from this state.

4.4 Performance of Basic Hill Climbing

Basic Hill Climbing Performance

Success Rate: Starting from random initialization, basic hill climbing solves only **14% of cases**.
Why So Low?

- Gets stuck in local minima frequently (86% of the time)
- No mechanism to escape local minima
- Random restarts would be needed to find solutions in most cases

5 Local Minima: The Main Challenge

5.1 What is a Local Minimum?

Local Minimum

A state s is a **local minimum** if:

$$h(s) \leq h(s') \quad \text{for all successors } s' \text{ of } s$$

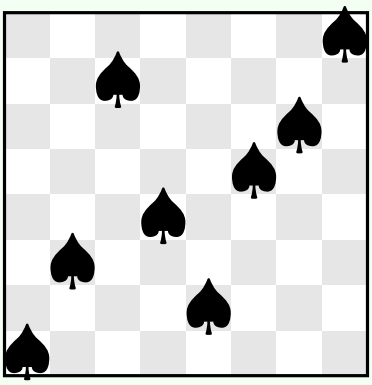
In other words, all neighbors of s have heuristic values that are greater than or equal to $h(s)$.
Types:

- **Strict Local Minimum**: $h(s) < h(s')$ for all successors s'
- **Local Minimum on Plateau**: $h(s) = h(s')$ for all successors s'

5.2 Example: Local Minimum in 8-Queens

8-Queens Local Minimum (From Lecture Slide 43)

Consider the following configuration (simplified representation):



Current State Analysis:

- Current h value is: **1** (one pair of queens attacking each other)
- To get $h = 0$, we must move one of the two queens involved in the conflict
- However, **every such move results in at least 2 new conflicts**
- Therefore, all neighbors have $h \geq 2$
- This state is a **local minimum** with $h = 1$

Why Is This Problematic?

- Hill climbing will stop at this state
- The returned "solution" has $h = 1$, which is not actually a solution
- The algorithm is stuck and cannot improve further
- Without modification, hill climbing fails to solve this instance

6 Difficulties in Local Search

Local search faces several fundamental challenges that limit its effectiveness.

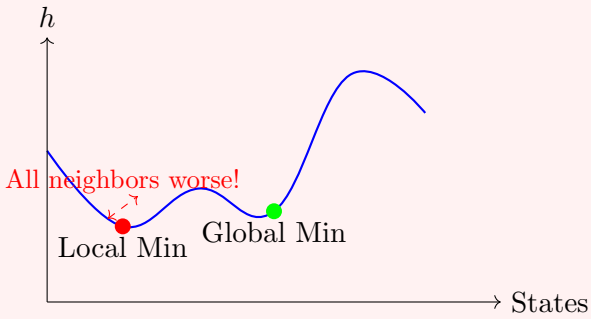
6.1 Local Minima

Problem: Local Minima

Definition: All neighbors look worse (have worse h values) than the current state.
Consequence:

- If we stop: The solution may be sub-optimal, or not even feasible
- If we don't stop: Where should we go next? All moves make things worse!

Illustration:



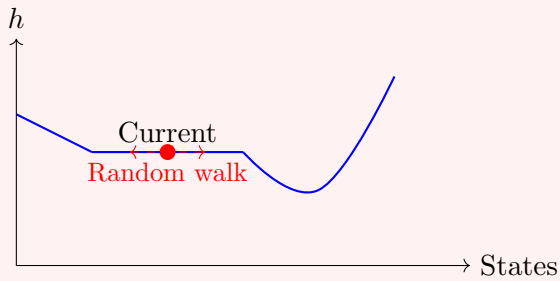
6.2 Plateaus

Problem: Plateaus

Definition: All neighbors have the same h value as the current state.
Consequence:

- Moves will be chosen completely at random
- No heuristic guidance available
- May wander aimlessly on the plateau
- Could potentially find a way off the plateau by chance

Illustration:



6.3 Ridges and Valleys

Additional Difficulties

Beyond local minima and plateaus, local search can also struggle with:
Ridges: Long, narrow elevated regions where:

- The optimal path follows along the ridge
- But each local step tends to move down off the ridge
- Local search has difficulty staying on the ridge

Valleys: Similar to ridges but inverted:

- The optimal path follows along the valley floor
- Local search may oscillate between valley walls
- Making progress along the valley is difficult

7 Strategies for Improving Local Search

Since basic hill climbing is incomplete and often gets stuck, several strategies have been developed to improve its effectiveness.

7.1 Random Restart

Random Restart Strategy

Idea: When reaching a local minimum, or after spending a certain amount of time without making progress, restart the search from a new random initial state.

Algorithm:

```
function HillClimbingWithRestart(heuristic h, max_restarts k)
    for i = 1 to k do
        n ← random state
        n ← HillClimbing(n, h)
        if h(n.State) = 0 then
            return n                                    ▷ Found solution
        end if
    end for
    return best state found across all restarts
end function
```

Rationale:

- Different starting points lead to different local minima
- With enough restarts, we may eventually start near the global minimum
- Each individual search is cheap (memory-efficient, fast)

Probabilistic Completeness: If we allow unlimited restarts and the state space is finite, we will eventually find a solution (if one exists) with probability 1.

7.2 Random Walk

Random Walk Strategy

Idea: Occasionally make a random move instead of always choosing the best neighbor. This can help escape local minima and traverse plateaus.

Algorithm Modification:

```
function HillClimbingWithRandomWalk(heuristic h, probability p)
    n ← InitNode()
    while true do
        Generate random number r ∈ [0, 1]
        if r < p then
            n′ ← random successor of n                    ▷ Random walk
        else
            n′ ← best successor of n                      ▷ Greedy step
        end if
        if stopping condition then
            return ExtractSolution(n)
        end if
        n ← n′
    end while
end function
```

Parameters:

- p: Probability of random walk (typically small, e.g., 0.1)
- Higher p: More exploration, less exploitation
- Lower p: More exploitation, less exploration

7.3 Sideways Moves

Sideways Move Strategy

Idea: Allow moves that do not improve the heuristic value (sideways moves) for a limited number of consecutive steps. This helps traverse plateaus.

Key Modification: Instead of stopping when $h(n') \geq h(n)$, continue if $h(n') = h(n)$, but count these sideways moves.

Algorithm Modification:

```
function HillClimbingWithSideways(heuristic h, max_sideways k)
    n ← InitNode()
    sideways_count ← 0
    while true do
        n′ ← best successor of n
        if h(n′) < h(n) then
            n ← n′
            sideways_count ← 0                          ▷ Reset counter on improvement
        else if h(n′) = h(n) and sideways_count < k then
            n ← n′
            sideways_count ← sideways_count + 1         ▷ Sideways move
        else
            return ExtractSolution(n)                   ▷ Stop
        end if
    end while
end function
```

Rationale:

- Allows traversing plateaus by making moves that don't improve h
- Limits consecutive sideways moves to avoid infinite loops
- Resets the counter when improvement is made

7.4 Parameter Configuration

The Configuration Challenge

Configuring these strategies requires setting many algorithm parameters:

- Number of random restarts
- Probability of random walk
- Maximum consecutive sideways moves
- Stopping conditions (time limits, iteration limits, etc.)

The Problem: Selecting good parameter values is a big issue in practice.
Approaches:

- **Domain Knowledge**: Use understanding of the problem to set reasonable values
- **Experimentation**: Try different values and measure performance
- **Meta-Optimization**: Use optimization techniques to tune the parameters themselves
- **Adaptive Methods**: Adjust parameters dynamically during search
- **Cross Your Fingers**: Hope the default values work!

8 Hill Climbing with Sideways Moves: 8-Queens Example

Let's see how adding sideways moves dramatically improves performance on the 8-Queens problem.

8.1 Configuration

Modified Algorithm for 8-Queens

Problem: 8-Queens (place 8 queens on board with no attacks)
State Representation: One queen per column
Heuristic: Number of pairs of queens attacking each other
Neighborhood: Move any queen within its column
Modification: Allow up to 100 consecutive moves where h value does not improve (sideways moves)
Initial State: Random placement

8.2 Performance Comparison

Algorithm	Success Rate	Notes
Basic Hill Climbing	14%	Stops at first local minimum
HC with Sideways Moves	94%	Allows up to 100 sideways moves

Dramatic Improvement

Key Insight: Allowing sideways moves increases success rate from 14% to 94%!
Why Such a Big Difference?

- Many "local minima" in basic hill climbing are actually shoulders of plateaus
- By allowing sideways moves, we can traverse these plateaus
- Often, traversing a plateau leads to a path that descends to a better region
- The limit of 100 consecutive sideways moves is enough to escape most plateaus, but not so large that we wander aimlessly

8.3 Exploration vs. Exploitation Trade-off

Exploration vs. Exploitation

Successful local search procedures often combine:
Exploration: Trying new, potentially suboptimal moves to discover better regions of the state space

- Random walks
- Sideways moves
- Random restarts

Exploitation: Following the heuristic to improve the current solution

- Always selecting the best neighbor
- Greedy descent

The Balance:

- Pure exploitation gets stuck in local minima
- Pure exploration is no better than random search
- The key is finding the right balance for the problem at hand

9 Can Local Minima Occur in Route Planning?

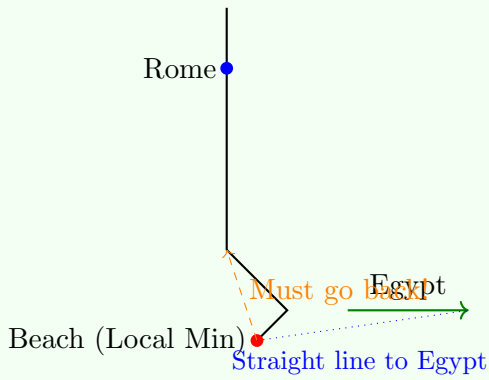
Let's examine whether local minima can occur in practical problems like route planning.

Question from Lecture Slide 46

Question: Can local minima occur in route planning with $h :=$ straight line distance?
Answer: **Yes**, local minima can occur.
Example - Rome to Egypt:

- Suppose we start in Rome and want to reach Egypt
- Using straight-line distance as heuristic
- Following hill climbing, we always take the road that decreases straight-line distance the most
- From Rome, roads going south initially decrease the distance to Egypt
- We might follow roads down the Italian peninsula
- Eventually, we reach a beach in southern Italy
- **Local Minimum**: From the beach, all roads lead back north (inland), which increases straight-line distance to Egypt!
- Hill climbing stops at the beach, even though we need to go back north, then east to actually reach Egypt

Visualization:



9.1 Similar Issues in Grid-Based Path Planning

Dead-End Street with Manhattan Distance

A similar problem occurs in grid-based path planning with obstacles (as shown in the lecture slides 15, 18-19):
Setup:

- Grid world with obstacles forming a "dead-end street"
- Using Manhattan distance as heuristic
- Goal is on the other side of the obstacles

Problem:

- Hill climbing follows Manhattan distance into the dead end
- At the end of the dead-end street, all moves increase Manhattan distance
- Local minimum is reached
- To actually reach the goal, must first go "backwards" (increase h) to get out of the dead end

Key Lesson

Admissible heuristics do not prevent local minima in hill climbing!
Even though straight-line distance and Manhattan distance are admissible heuristics:

- They work well for systematic search (A*)
- They can still lead to local minima in hill climbing
- The issue is that hill climbing only considers immediate neighbors
- It cannot "see ahead" to know that a temporary increase in h leads to a better path

10 Comparison: Local Search vs. Systematic Search

10.1 Key Differences

Aspect	Systematic Search (e.g., A*)	Local Search (e.g., Hill Climbing)
Memory Usage	$O(b^d)$ - exponential	$O(b)$ - linear
Completeness	Yes (with proper conditions)	No
Optimality	Yes (with admissible h)	No
Decision Making	Global (considers entire frontier)	Local (considers only neighbors)
Backtracking	Yes (via open list)	No
State Revisiting	Prevented (via closed set)	Naturally avoided (always improves)
Typical Use Cases	Finding optimal paths	Optimization, large state spaces
Performance	Guaranteed solution if exists	May get stuck, but very fast

10.2 When to Use Each Approach

Systematic Search - Use When:

- Optimal solution is required
- Completeness is necessary (must find solution if it exists)
- State space is manageable in size
- Memory is available
- Can afford the computational cost
- Good admissible heuristic is available

Examples: Robot path planning, puzzle solving, route planning in small graphs

Local Search - Use When:

- State space is very large (systematic search infeasible)
- Memory is severely limited
- Sub-optimal solution is acceptable
- All states are valid solutions (optimization problem)
- Quick solutions are needed
- Can afford multiple random restarts

Examples: Scheduling problems, configuration problems, parameter tuning, n-Queens for large n

10.3 Hybrid Approaches

Combining Systematic and Local Search

In practice, hybrid approaches can leverage strengths of both:
Iterative Improvement After Systematic Search:

- Use A* or other systematic method to find initial solution
- Use local search to improve the solution further

Systematic Search with Local Search Heuristics:

- Use local search to quickly find approximate goal distances
- Use these as heuristic values for A*

Restarting with Memory:

- Keep track of visited local minima
- When restarting, avoid previously visited local minima
- Gradually builds a more complete picture of state space

11 Advanced Local Search Methods

While hill climbing is the most basic local search method, many more sophisticated variants have been developed.

11.1 Simulated Annealing

Simulated Annealing

Idea: Inspired by the physical process of annealing metals. Allow occasional "uphill" moves (moves that worsen h) with probability that decreases over time.
Key Concept - Temperature:

- Start with high "temperature" T
- Accept uphill moves with probability $e^{-\Delta h/T}$ where $\Delta h = h(n') - h(n) > 0$
- Gradually decrease T over time (cooling schedule)
- At high T: Nearly random walk (lots of exploration)
- At low T: Nearly hill climbing (mostly exploitation)

Advantages:

- Can escape local minima (early in search)
- Provably converges to global optimum (with right cooling schedule)
- Single parameter to tune (cooling schedule)

11.2 Tabu Search

Tabu Search

Idea: Maintain a "tabu list" of recently visited states. Forbid moves that would return to tabu states.
Key Features:

- Keeps short-term memory of recent states (tabu list)
- Prevents cycling back to recently visited states
- Can accept uphill moves if necessary (when all neighbors are worse)
- More aggressive than hill climbing in exploring state space

Variations:

- Tabu tenure: How long a state remains tabu
- Aspiration criteria: Conditions for overriding tabu status
- Long-term memory: Track frequently visited regions to diversify search

11.3 Genetic Algorithms and Population-Based Methods

Population-Based Local Search

Instead of maintaining a single current state, maintain a **population** of states:
Genetic Algorithms:

- Population of candidate solutions
- Selection: Choose better solutions more often
- Crossover: Combine parts of two solutions
- Mutation: Random modifications
- Iterate to evolve better solutions

Advantages:

- Explores multiple regions of state space simultaneously
- Can escape local minima through population diversity
- Crossover can combine good features from different solutions

Disadvantages:

- Higher memory usage (multiple states)
- More complex (many parameters to configure)
- Problem-dependent (how to represent states, crossover, etc.)

12 Summary and Key Takeaways

12.1 Main Concepts

Local Search - Summary

Core Idea: Make decisions based solely on immediate neighbors of the current state.
Hill Climbing Algorithm:

- Start from initial state
- Repeatedly move to best neighbor
- Stop when no neighbor is better (local minimum)

Properties:

- **Incomplete**: Not guaranteed to find solution
- **Not Optimal**: May find suboptimal solution
- **Fast**: Makes quick decisions
- **Memory Efficient**: Only $O(b)$ space

Main Challenge: Gets stuck in local minima

12.2 Key Difficulties

1. **Local Minima**: States where all neighbors are worse
   - Algorithm stops even though better solutions exist
   - Very common in practice
2. **Plateaus**: Regions where all states have same heuristic value
   - No heuristic guidance
   - Random walk required to escape
3. **Ridges/Valleys**: Difficult terrain in the search landscape
   - Optimal path is narrow
   - Local steps tend to move away from optimal path

12.3 Improvement Strategies

1. **Random Restart**: Try multiple starting points
   - Different starts lead to different local minima
   - Eventually may start near global optimum
2. **Sideways Moves**: Allow moves that don't improve h
   - Helps traverse plateaus
   - Limit consecutive sideways moves to avoid infinite loops
3. **Random Walk**: Occasionally move randomly
   - Helps escape local minima
   - Balances exploration and exploitation
4. **Advanced Methods**: Simulated annealing, tabu search, genetic algorithms

12.4 Practical Success

When Local Search Works Well

Despite theoretical limitations, local search is very successful in practice:
8-Queens Example:

- Basic hill climbing: 14% success rate
- With sideways moves: 94% success rate

Why It Works:

- Many real-world problems have relatively few local minima
- Even if many local minima exist, many are close to global optimum
- Random restart combined with improvements often finds good solutions quickly
- Memory efficiency allows scaling to very large problems

Applications:

- Scheduling and timetabling
- Circuit board layout
- Network configuration
- Parameter optimization
- Machine learning (gradient descent)

12.5 Comparison with Systematic Search

Property	Systematic (A*)	Local (Hill Climbing)
Complete	Yes	No
Optimal	Yes (with admissible h)	No
Memory	$O(b^d)$	$O(b)$
Speed	Slower (explores systematically)	Faster (greedy decisions)
Use Case	Optimal paths needed	Optimization, large spaces

12.6 Final Thoughts

Exploration vs. Exploitation

The Fundamental Trade-off:
Successful local search requires balancing:

- **Exploitation**: Following the heuristic to improve current solution
- **Exploration**: Trying new areas to find better solutions

Pure exploitation → Gets stuck in local minima
Pure exploration → Random search (no guidance)
Good balance → Efficient search that often finds good solutions
This trade-off is fundamental not just in local search, but in many areas of AI and machine learning!